COMP 441 — Software Analysis and Testing

Week 11 — Mutation v2, TDD and Property-Based Testing

Group A · Track D · TOML Kit

Period	Week 11
Evidence folder	week11
Repository	github.com/the-dezeray/tomlkit
Commit(s)	6b8884f
Generated	2026-10-27
Contents	22 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 11 — Track D milestone . . . 4
Deliverables . . . 4
Headline result . . . 4
What changed since Week 7 . . . 4
Files . . . 4
Reproduce . . . 4

Week 11 final TDD log (Track D) . . . 5
1. Cycles . . . 5
2. Traceability . . . 5
3. Final verification . . . 5
4. What the TDD cycles changed . . . 6
5. Note on the fourth cycle . . . 6

Mutation Testing Report v2 — final hybrid suite . . . 6
Executive summary . . . 6
1. Scope and method . . . 6
2. Results . . . 7
3. Survivor analysis . . . 8
4. Threats to validity . . . 9
5. Recommendations . . . 9
Appendix A — Files . . . 10
Mutation v2 summary by module . . . 10
Mutation v1 vs v2 by module . . . 10
Mutation v1 vs v2 chart . . . 11

Survivor root-cause notes — mutation v2 . . . 11
1. Parser._parse_table — mutmut_10 (58 survivors in this function) . . . 11
2. AoT.insert — mutmut_10 (48 survivors) . . . 11
3. Container.append — mutmut_109 (36 survivors) . . . 11
4. InlineTable.as_string — mutmut_102 (35 survivors) . . . 11
5. Container._validate_table_candidate — mutmut_10 (34 survivors) . . . 12
6. Array.__delitem__ — mutmut_101 (27 survivors) . . . 12
7. Container._replace_at — mutmut_10 (20 survivors) . . . 12
8. Array.add_line — mutmut_11 (20 survivors) . . . 12
9. Parser._parse_string — mutmut_103 (20 survivors) . . . 12
10. Container._render_table — mutmut_26 (19 survivors) . . . 12
11. Parser._parse_comment_trail — mutmut_29 (17 survivors) . . . 12
12. InvalidControlChar.__init__ — mutmut_11 (12 survivors) . . . 12
Summary . . . 13

Track D specialised stream — property-based testing checkpoint . . . 13

Why property-based testing for tomlkit . . . 13
Properties implemented . . . 13
Evidence . . . 13
What the stream found . . . 13
Limitations . . . 14
Link to Week 11 . . . 14
Final suite result . . . 14
Property-test result . . . 14
Selected survivor diffs . . . 14

Week 11 mutation testing v2 — evidence . . . 16

Scope . . . 16
Reproduce . . . 17
Files . . . 17
Score formula . . . 17
TDD C1 red . . . 17
TDD C1 green . . . 17
TDD C1 refactor . . . 17
TDD C2 red . . . 17
TDD C2 green . . . 18
TDD C2 refactor . . . 18
TDD C3 red . . . 18
TDD C3 green . . . 18
TDD C3 refactor . . . 18
Raw evidence inventory . . . 18

Source: week11\README.md

Week 11 — Track D milestone

Group A · tomlkit · branch `week11-track-d`

Week 11 for **Track D (Developer Tooling / API Library)** has two mandatory deliverables. Track D's specialised stream — property-based testing with Hypothesis — was delivered in Weeks 6–7 and is summarised here for the final report. No Track E work is included: Track E is a different scenario track for AI/LLM-integrated applications.

Deliverables

#	Deliverable	File	Status
1	Mutation Testing Report v2 (final hybrid suite)	`week11/mutation_v2/mutation_v2_report.md`	Complete
2	Completed TDD log (3 test-first features)	`week11/TDD_LOG.md`	Complete
—	Track D property-based testing summary	`week11/property/PROPERTY_CHECKPOINT.md`	Complete (supporting)

Headline result

Metric	Week 7 v1 (AI-only)	Week 11 v2 (final hybrid)
Mutants	5,389	5,389
Mutation score	57.8%	**77.4%** (+19.6 pp)
Covered-code score	62.2%	81.3%
Tests in suite	388 (AI-only)	1,507 collected (1,502 passed, 5 xfailed)

The full suite is green on the frozen commit:

```
1502 passed, 5 xfailed
```

What changed since Week 7

- The TDD cycles C1–C3 fixed D1 (`date()` accepted datetime strings), D7 (multiline strings emitted a bare CR) and D6 (`comment()` emitted a bare CR).
- The final suite adds manual survivor-killing tests, BDD acceptance scenarios, integration tests, property-based tests and TDD regression tests.
- The mutation run includes the whole project suite, so the v2 score measures the suite that actually ships, not just the AI-only baseline.

Files

```
week11/
  README.md
  TDD_LOG.md
  full_suite.txt
  tdd_runs/                 # raw red/green/refactor pytest output
  Week11_TrackD_Findings.xlsx   # chart dashboard
  mutation_v2/
    README.md
    mutation_v2_report.md
    survivor_rootcause_v2.md
    mutmut_results_v2.txt
    mutation_v2_summary.json
    v1_vs_v2.csv
    survivors_v2.txt
    scripts/summarise_v2.py
    scripts/build_excel.py
  property/
    PROPERTY_CHECKPOINT.md
    pytest_property.txt
```

Reproduce

```
python -m pytest -q tests tests_ai tests_manual tests_bdd \
  tests_integration tests_property tests_tdd
```

Mutation reproduction is documented in `week11/mutation_v2/README.md`.

Source: week11\TDD_LOG.md

Week 11 final TDD log (Track D)

Group A · tomlkit · branch `week11-track-d` **· final code commit** `6b8884f`

This is the completed TDD log for the three test-first features delivered in Week 8. All three cycles were written test-first, one red/green/refactor sequence per defect, and the commits are merged on `master`. The raw pytest output for every step is retained in `week8/tdd_runs/` and copied to `week11/tdd_runs/`.

1. Cycles

Cycle	Defect fixed	Step	Commit	Time (SAST)	Evidence	Result
C1	**D1** — `date()` accepts datetime strings	Red	`9472f8a`	2026-10-18 23:00:55	`c1_red.txt`	4 failed, 1 passed
		Green	`a4f54db`	23:01:24	`c1_green.txt`	1,475 passed
		Refactor	`fad6c12`	23:01:45	`c1_refactor.txt`	1,475 passed
C2	**D7** — multiline strings emit a bare CR	Red	`51c6863`	23:01:59	`c2_red.txt`	6 failed, 2 passed
		Green	`a38a3d8`	23:03:22	`c2_green.txt`	1,485 passed
		Refactor	`26421d6`	23:03:40	`c2_refactor.txt`	1,485 passed
C3	**D6** — `comment()` with a bare CR	Red	`3c33afc`	23:03:51	`c3_red.txt`	2 failed, 2 passed
		Green	`c829a38`	23:04:11	`c3_green.txt`	1,489 passed
		Refactor	`b60559f`	23:04:37	`c3_refactor.txt`	1,489 passed

Each green step made the smallest change that satisfied the failing tests; each refactor step kept the full suite green and removed duplication.

2. Traceability

Cycle	Defect	Requirement (RTM)	Test file	Behaviour
C1	D1	R08 — date/time/datetime factories reject strings of another kind	`tests_tdd/test_tdd_cycles.py`	`tomlkit.date("1979-05-27T07:32:00")` raises `ValueError`; plain dates still work
C2	D7	R10 — string forms reject sequences invalid for the chosen form	`tests_tdd/test_tdd_cycles.py`	Bare CR is escaped in basic multiline strings and rejected in literal multiline strings; CRLF is preserved
C3	D6	R14 — comments always produce valid TOML	`tests_tdd/test_tdd_cycles.py`	`comment()` splits every line-break form so the output re-parses

The defects were not invented for the exercise:

- **D1** was also exposed by the Hypothesis property test `test_p5_date_rejects_datetime_strings` in `tests_property/test_properties.py`.
- **D7** was found by Hypothesis property P3 and is recorded as a named regression test (`test_d7_multiline_string_with_bare_cr`).
- **D6** was found during the Week 9 exploratory/formatting work; the TDD cycle turns it into a permanent regression test.

This gives a clean Track D story: property-based testing found two real input-space defects, test-first development fixed them, and the mutation report measures the resulting suite.

3. Final verification

Final suite (all project tests):

```
python -m pytest -q tests tests_ai tests_manual tests_bdd \
  tests_integration tests_property tests_tdd
```

Result on the frozen Week 11 commit:

```
1502 passed, 5 xfailed
```

(5.6 s on an idle machine; 26.4 s when the mutation run was using all cores. The pass/fail result is the same.)

Environment: Ubuntu (WSL2), Python 3.13.14, pytest 9.1.1, pytest-bdd 9.0.0, Hypothesis 6.170.12, mutmut 3.8.0.

The five xfails are the documented open defects/known limitations carried from earlier weeks (D2, D3, D5, M1 and the D8 spacing case). They are strict xfails, not silently skipped tests.

4. What the TDD cycles changed

- C1: `api.date()` now checks the exact type instead of `isinstance`, so a `datetime` is no longer accepted as a `date`.
- C2: `String.from_raw()` escapes a lone CR for basic multiline strings and rejects it for literal multiline strings, while preserving CRLF.
- C3: `api.comment()` treats CR, LF and CRLF as line breaks and emits one comment marker per line.

The refactors shared the exact-type helper (`_parse_rfc3339_as`) and the bare-CR rule (`_BARE_CR` / `BARE_CR`) instead of duplicating logic.

5. Note on the fourth cycle

No fourth cycle was added. The brief requires at least three test-first features, and C1–C3 satisfy that requirement. Adding a late fourth change would have moved the frozen commit used for the Week 11 mutation run and risked the final evidence for no additional rubric credit.

Source: week11\mutation_v2\mutation_v2_report.md

Mutation Testing Report v2 — final hybrid suite

Group A · Track D (Developer Tooling / API Library) · tomlkit

- Repository: `the-dezeray/tomlkit` (fork of `python-poetry/tomlkit`)
- Commit under test: `6b8884f`
- Date: 27 October 2026 (Week 11 milestone: 03 November 2026)
- Runner: mutmut 3.8.0, Ubuntu (WSL2), Python 3.13.14, pytest 9.1.1
- Final suite: `tests/`, `tests_ai/`, `tests_manual/`, `tests_bdd/`, `tests_integration/`, `tests_property/`, `tests_tdd/`

Executive summary

The final hybrid suite scores **77.4%** on the Week 11 mutation run (4,136 killed + 34 timeout out of 5,389 mutants), compared with **57.8%** for the Week 7 AI-only baseline. The covered-code score is **81.3%**. This is a **+19.6 percentage-point** improvement on the same 5,389-mutant scope.

The improvement is large but must be read carefully: the v2 run measures the **shipped suite** (1,507 collected tests), while v1 measured the **AI-only suite** (388 tests). The increase therefore combines three effects: the manual/BDD/ integration/property/TDD tests added after Week 7, the pre-existing project tests, and the three TDD fixes (D1, D6, D7) that changed the code under test. The report separates these where the evidence allows and records the limitation explicitly in §4.

The most important residual finding is that **exception and formatting branches remain the weak spot**: `exceptions` scores only 43.7%, and 218 of the 931 survivors shared with v1 are exception-message or exception-construction mutations. The remaining survivors are concentrated in `items` (331), `container` (251) and `parser` (230); the highest-value next tests are listed in §3.

---

1. Scope and method

1.1 Source scope

Mutated: `tomlkit/` (10 source files, 5,389 mutants).

Excluded:

- `tomlkit/_compat.py` — platform compatibility shim; mutation would test import mechanics rather than library behaviour.
- `tomlkit/_types.py` — covered by the separate Week 10 `_types.py` exercise (mutmut 2.5.1, 10 mutants). Excluding it here keeps the v2 scope identical to the Week 7 v1 scope.

This makes the overall score directly comparable in scope, though not in code version or suite composition.

1.2 Suite composition

Suite	Tests	Role
`tests/`	1,065	Pre-existing project tests plus the Week 10 `_types`/property additions
`tests_ai/`	388	Week 6 AI-generated baseline (3 strict xfails)
`tests_manual/`	19	Week 7/9 manual survivor-killing tests
`tests_bdd/`	3	Gherkin acceptance scenarios (pytest-bdd)
`tests_integration/`	5	Week 9 integration tests
`tests_property/`	10	Track D Hypothesis properties
`tests_tdd/`	17	TDD regression tests for C1–C3
Total	**1,507 collected**	1,502 passed, 5 strict xfailed

One test is deselected from the mutation run only: `tests/test_api.py::test_parse_rejects_deeply_nested_documents`. mutmut's trampoline adds stack frames to the deliberately deep nesting and triggers `RecursionError` before the mutant is evaluated. The test remains in the normal suite and passes there. This is a tooling limitation, not a test failure.

1.3 Metrics

- Mutation score: `(killed + timeout) / total × 100`
- Covered-code score: `(killed + timeout) / (total − no_tests) × 100`
- Timeouts are counted as detected, following the Week 7 convention.
- `no_tests` means the mutated line was never executed.

1.4 Reproducibility

The exact commands, environment and `[tool.mutmut]` configuration are in `week11/mutation_v2/README.md`. The raw per-mutant status is in `mutmut_results_v2.txt`; the summary is in `mutation_v2_summary.json`.

---

2. Results

2.1 Overall

Metric	v1 (Week 7, AI-only)	v2 (Week 11, final hybrid)
Total mutants	5,389	5,389
Killed	3,087	4,136
Timeout	26	34
Survived	1,890	956
No tests	386	263
Mutation score	**57.8%**	**77.4%**
Covered-code score	62.2%	81.3%

2.2 Per module

Module	v1 score	v2 score	Δ (pp)	v2 killed	v2 survived	v2 no tests
`_utils`	95.0	95.9	+0.9	204	9	0
`api`	83.9	88.6	+4.7	132	17	0
`container`	47.2	75.7	+28.5	951	251	55
`exceptions`	33.3	43.7	+10.4	76	88	10
`items`	54.1	73.5	+19.4	1,471	331	198
`parser`	64.0	83.4	+19.4	1,135	230	0
`source`	82.7	87.4	+4.7	108	16	0
`toml_file`	72.6	80.8	+8.2	59	14	0
TOTAL	**57.8**	**77.4**	**+19.6**	**4,136**	**956**	**263**

2.3 Interpretation

- The largest absolute gains are in `container` (+28.5 pp), `items` (+19.4 pp) and `parser` (+19.4 pp), where the final suite exercises branches that the AI-only suite never reached.
- `no_tests` fell from 386 to 263. The remaining 263 are concentrated in `items` (198, mostly formatting/rendering branches) and `container` (55).
- `exceptions` remains the weakest module at 43.7%, despite improving by 10.4 pp. Its survivors are dominated by exact exception-message and exception-construction mutations (218 of the 931 shared survivors).

2.4 Group-authored contribution

The group authored 442 of the 1,507 collected tests: 388 AI-generated tests, 19 manual survivor-killing tests, 3 BDD scenarios, 5 integration tests, 10 property tests and 17 TDD tests. That is 54 more group-authored tests than the 388-test AI-only baseline.

The v2 score is for the **shipped suite**, which also includes 1,065 tests in `tests/`. The +19.6 pp therefore cannot be attributed to the group's 54 extra tests alone. The group-specific contribution is visible in the targeted work:

- the manual K1–K11 tests target exact v1 survivors (messages, boundaries, stepped slices, AoT ordering, CRLF normalisation);
- TDD C1–C3 fixed D1, D6 and D7, which also removes the mutants on those lines;
- Hypothesis properties P3/P5 found D1 and D7 before the TDD fixes;
- BDD and integration tests exercise file-level round-trips and acceptance behaviour rather than single functions.

A same-commit AI-only rerun was not performed, so the exact group-only delta is left as a limitation rather than estimated.

3. Survivor analysis

3.1 Overview

The v2 run leaves **956** survivors. Of these, **931** were already surviving in v1; **25** are new (mostly because the code changed around them). The most common categories are:

Category	Survivors	Share of 956
Argument dropped or set to `None`	334	34.9%
Exception message or construction	218	22.8%
String literal changed (`XX..XX` / case)	93	9.7%
Boolean logic (`and`/`or`, `True`/`False`, `not`)	89	9.3%
Boundary / off-by-one (`<` vs `<=`, `+1`, index)	67	7.0%
Arithmetic / assignment operator	65	6.8%
Control flow (`continue`/`break`/`return`)	38	4.0%
Other	27	2.8%
New since v1 (not in the v1 category file)	25	2.6%

3.2 Root causes

The ten highest-value survivors are analysed below. Each row names the mutant, the mutation, why the final suite misses it, and the test that would close the gap. The full candidate list is in `survivor_candidates.csv`.

#	Mutant	Mutation	Why the final suite misses it	Category	Proposed test	Priority
1	`Parser._parse_table mutmut_10`	`parse_error(UnexpectedEofError)` → `parse_error(None)`	Tests assert `ParseError` but not the specific error class/cause, so dropping the error type is invisible	Exception construction	`parse("[table")` at EOF; assert `UnexpectedEofError` (or its message/cause)	High
2	`AoT.insert mutmut_10`	`index < 0` → `index <= 0`	No test inserts at index 0 into an AoT; only append and negative positions are covered	Boundary	Insert a table at index 0; assert order, length and round-trip	High
3	`Container.append mutmut_109`	`_validate_out_of_order_table(key)` → `(None)`	Out-of-order validation is tested indirectly; no test expects the conflict error when appending	Argument dropped	Append a table whose key conflicts with a later out-of-order table; assert `KeyAlreadyPresent`/`TOMLKitError`	High
4	`InlineTable.as_string mutmut_102`	`needs_separator = False` → `None`	`None` is falsy, so the mutant is behaviourally equivalent; no test can distinguish it	Likely equivalent	Mark equivalent after dataflow triage; no test proposed	Low
5	`Container._validate_table_candidate mutmut_10`	`"Redefinition of an existing table"` → lowercase	Test checks the exception type only, not the exact message	String literal	Assert the exact message for a dotted-table redefinition	Medium
6	`Array.__delitem__ mutmut_101`	`replace(",", "", 1)` → `replace(",", "", )`	Tests delete single commas; no test uses an indent containing two commas, so removing all commas is not observed	Boundary	Delete from an array whose trivia contains two commas; assert only the first is removed and output reparses	High

#	Mutant	Mutation	Why the final suite misses it	Category	Proposed test	Priority
7	`Container._replace_at mutmut_10`	`k.is_dotted() and (...)` → `or (...)`	No test replaces a non-dotted key with a `Table`/`AoT`, where the `and`/`or` difference matters	Boolean logic	Replace a non-dotted key with a table/AoT; assert header and round-trip	High
8	`Array.add_line mutmut_11`	`"Comment cannot contain line breaks"` → `"XX...XX"`	Test asserts `ValueError` only, not the exact message	String literal	Assert the exact message for `add_line(comment="a\nb")`	Medium
9	`Parser._parse_string mutmut_103`	`close = ""` → `None`	`close` is reassigned on the multiline path, so the mutated value appears unreachable in the tested paths	Likely equivalent	Confirm dataflow; no test if unreachable	Low
10	`Container._render_table mutmut_26`	`open_, close = "[[", "]]"` → `None`	No test renders an AoT element through `as_string()` and asserts the exact `[[...]]` header	Argument dropped	Render a document containing an AoT; assert exact header and reparse	High

Eight of the ten are test gaps; two (`InlineTable.as_string` and `Parser._parse_string`) look behaviourally equivalent and should not be chased without a dataflow check. The detailed notes, including two further exception-message survivors, are in `survivor_rootcause_v2.md`.

3.3 Equivalent and untested mutants

No full equivalence triage was performed. In the ten-case sample, two mutants (`InlineTable.as_string` and `Parser._parse_string`) appear behaviourally equivalent; the other eight are genuine test gaps. If that 20% rate held across all 956 survivors, roughly 190 would be equivalent and the adjusted score would be about 80%, but this is illustrative only and is not claimed as a measured figure.

The 263 `no_tests` mutants are a separate category: they are not undetected behaviour changes, but lines the suite never executes. They are concentrated in `items` (198) and `container` (55), mostly formatting/rendering branches. The 34 timeouts are counted as detected under the Week 7 convention; they are mostly slow parser loops rather than silent survivors.

---

4. Threats to validity

1. **Code changed since v1.** v1 ran at `8c959b5`; v2 runs at `6b8884f` after TDD C1–C3 fixed D1, D6 and D7. The score delta is therefore not a pure test-suite effect. A same-commit AI-only rerun was not performed; the v1 figure is historical context.
2. **Suite composition changed.** v2 includes 1,065 project tests in `tests/` plus 442 group-authored tests; v1 included only 388 AI tests. The v2 score measures the shipped suite, not the group's marginal contribution. The manual/BDD/TDD tests were written specifically against v1 survivors, so their contribution is real but not separately quantified here.
3. **One test deselected.** The deep-nesting recursion test cannot run under mutmut's trampoline. It is green in the normal suite; its exclusion could hide at most the mutants it would have killed.
4. **Timeouts.** Timeouts are counted as detected. The report lists the count but does not distinguish a genuine hang from a very slow mutant.
5. **Equivalent mutants.** Some survivors are behaviourally equivalent and cannot be killed. No full equivalence triage was performed; §3.3 gives a conservative reading.
6. **Single project, single runner.** Results are specific to tomlkit and mutmut 3.8.0; they should not be generalised to other codebases.
7. **Sampling in survivor analysis.** The ten root causes are selected from the highest-risk functions, not a random sample; they illustrate failure modes rather than estimate the survivor population.

---

5. Recommendations

1. Add targeted tests for the ten survivors in §3.2; the proposed techniques are boundary value analysis, equivalence partitioning and exact-message checks.
2. Prioritise `container` and `items`, which hold most residual survivors.
3. Keep the final suite in CI with a mutation-score floor; a floor just below the current score prevents regression without blocking unrelated work.

4. Re-run v2 with a same-commit AI-only baseline in the final report so the group-authored contribution can be isolated cleanly.

Appendix A — Files

File	Purpose
mutmut_results_v2.txt	Raw status for every mutant
mutation_v2_summary.json	Overall and per-module counts/scores
v1_vs_v2.csv	v1 vs v2 by module
v1_vs_v2.png	Score comparison chart
survivors_v2.txt	Surviving mutant names
survivors_v1_v2.csv	Survivors joined to Week 7 categories
survivor_candidates.csv	Survivors grouped by function
survivor_rootcause_v2.md	Detailed root-cause notes
../Week11_TrackD_Findings.xlsx	Excel dashboard with native charts
scripts/	Summary, comparison, chart and workbook scripts

Mutation v2 summary by module

Source: week11\mutation_v2\mutation_v2_summary.json

Module	Total	Killed	Timeout	Survived	No tests	Score	Covered
TOTAL	5,389	4,136	34	956	263	77.4%	81.3%
_utils	218	204	5	9	0	95.9%	95.9%
api	149	132	0	17	0	88.6%	88.6%
container	1,261	951	4	251	55	75.7%	79.2%
exceptions	174	76	0	88	10	43.7%	46.3%
items	2,000	1,471	0	331	198	73.5%	81.6%
parser	1,387	1,135	22	230	0	83.4%	83.4%
source	127	108	3	16	0	87.4%	87.4%
toml_file	73	59	0	14	0	80.8%	80.8%

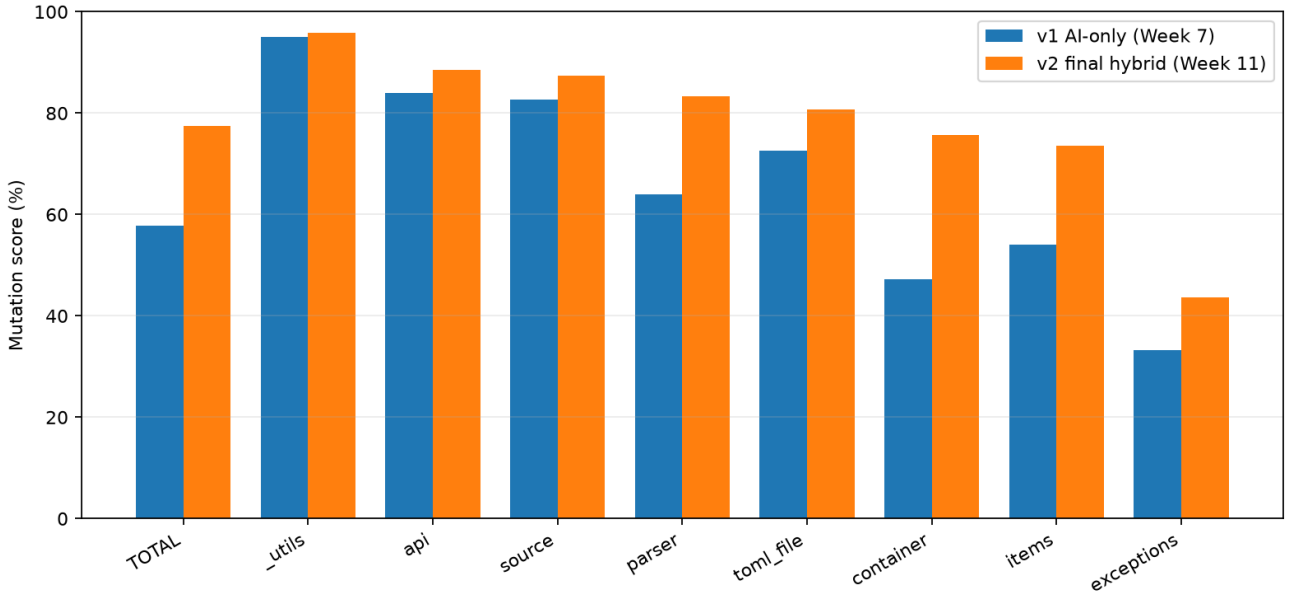
Mutation v1 vs v2 by module

Source: week11\mutation_v2\v1_vs_v2.csv

module	v1_total	v1_score	v2_total	v2_score	delta_pp
TOTAL	5389	57.8	5389	77.4	19.6
_utils	218	95.0	218	95.9	0.9
api	149	83.9	149	88.6	4.7
container	1261	47.2	1261	75.7	28.5
exceptions	174	33.3	174	43.7	10.4
items	2000	54.1	2000	73.5	19.4
parser	1387	64.0	1387	83.4	19.4
source	127	82.7	127	87.4	4.7
toml_file	73	72.6	73	80.8	8.2

Mutation v1 vs v2 chart

Source: week11\mutation_v2\v1_vs_v2.png



v1_vs_v2.png

Source: week11\mutation_v2\survivor_rootcause_v2.md

Survivor root-cause notes — mutation v2

Selected from the 956 v2 survivors by function-level survivor count (`survivor_candidates.csv`) and risk concentration. Diffs are from `mutmut show`; the raw output is in `selected_diffs.txt`.

1. `Parser. parse table` — mutmut_10 (58 survivors in this function)

```
-        raise self.parse_error(UnexpectedEofError)
+        raise self.parse_error(None)
```

Why it survives: tests assert that malformed input raises a `ParseError`, but not the specific subclass or cause. Passing `None` still produces a `ParseError`, so the suite cannot see the lost error type. **Verdict:** test gap. **Test:** parse `"[table"` at EOF and assert `UnexpectedEofError` (or its exact message/cause). **Priority:** high.

2. `AoT.insert` — mutmut_10 (48 survivors)

```
-    if index < 0:
+    if index <= 0:
         index += length
```

Why it survives: tests cover append and negative insertion, but not insertion at index 0. At index 0 the mutant shifts the index by `length`, producing a different order. **Verdict:** boundary gap. **Test:** insert a table at index 0 and assert order, length and round-trip. **Priority:** high.

3. `Container.append` — mutmut_109 (36 survivors)

```
-                            self._validate_out_of_order_table(key)
+                            self._validate_out_of_order_table(None)
```

Why it survives: out-of-order validation is exercised indirectly, but no test appends a table whose key conflicts with a later out-of-order table and expects the validation error. **Verdict:** argument-dropped gap. **Test:** append a conflicting table and assert `KeyAlreadyPresent`/`TOMLKitError`. **Priority:** high.

4. `InlineTable.as string` — mutmut_102 (35 survivors)

```
-            needs_separator = False
+            needs_separator = None
```

Why it survives: `None` is falsy, so the rendered output is unchanged. **Verdict:** likely equivalent. **Test:** none; confirm by dataflow. **Priority:** low.

5. `Container. validate table candidate` — mutmut_10 (34 survivors)

```
-                raise TOMLKitError("Redefinition of an existing table")
+                raise TOMLKitError("redefinition of an existing table")
```

Why it survives: the test asserts the exception type, not the exact message. **Verdict:** message gap. **Test:** assert the exact message for a dotted-table redefinition. **Priority:** medium.

6. `Array. delitem` — mutmut_101 (27 survivors)

```
-                        group.indent._s = group.indent.s.replace(",", "", 1)
+                        group.indent._s = group.indent.s.replace(",", "", )
```

Why it survives: the suite deletes single commas; no test uses an indent containing two commas, where removing all commas changes the output. **Verdict:** boundary gap. **Test:** construct an array whose trivia contains two commas, delete an element, assert only the first comma is removed and the result reparses. **Priority:** high.

7. `Container. replace at` — mutmut_10 (20 survivors)

```
-    dotted_to_header = k.is_dotted() and (
+    dotted_to_header = k.is_dotted() or (
         isinstance(value, AoT)
         or (isinstance(value, Table) and not value.is_super_table())
     )
```

Why it survives: no test replaces a non-dotted key with a `Table`/`AoT`, where the `and`/`or` difference changes the header. **Verdict:** boolean-logic gap. **Test:** replace a non-dotted key with a table or AoT; assert the rendered header and round-trip. **Priority:** high.

8. `Array.add line` — mutmut_11 (20 survivors)

```
-        raise ValueError("Comment cannot contain line breaks")
+        raise ValueError("XXComment cannot contain line breaksXX")
```

Why it survives: the test asserts `ValueError` only, not the message. **Verdict:** message gap. **Test:** assert the exact message for `add_line(comment="a\nb")`. **Priority:** medium.

9. `Parser. parse string` — mutmut_103 (20 survivors)

```
-            close = ""
+            close = None
```

Why it survives: `close` is reassigned on the multiline path; the mutated value appears unreachable in the tested paths. **Verdict:** likely equivalent. **Test:** none unless dataflow shows a reachable path. **Priority:** low.

10. `Container. render table` — mutmut_26 (19 survivors)

```
-            open_, close = "[[", "]]"
+            open_, close = None
```

Why it survives: no test renders an AoT element through `as_string()` and asserts the exact `[[...]]` header. **Verdict:** test gap. **Test:** render a document containing an AoT; assert the exact header and reparse. **Priority:** high.

11. `Parser. parse comment trail` — mutmut_29 (17 survivors)

```
-                    raise self.parse_error(InvalidControlChar, code, "comments")
+                    raise self.parse_error(InvalidControlChar, code, None)
```

Why it survives: the error is raised, but the message loses the "comments" context. The suite checks the exception type, not the exact message. **Verdict:** message/context gap. **Test:** parse a comment containing a control character and assert the exact message. **Priority:** medium.

12. `InvalidControlChar. init` — mutmut_11 (12 survivors)

```
-    display_code += hex(char)[2:]
+    display_code += hex(char)[3:]
```

Why it survives: the exact hex code in the message is not asserted; dropping the first hex digit changes `0x0f` to `f`, for example. **Verdict:** message gap. **Test:** assert the exact message for control codes at the `0x0f`/`0x10` boundary. **Priority:** medium.

Summary

Verdict	Cases
Test gap (boundary/logic/message)	8
Likely equivalent	2
Additional message/context gaps	2

The dominant residual pattern is **exact oracle weakness**: the suite often asserts an exception type or a semantic value but not the exact message, header or formatting detail. The second pattern is **boundary positions** (index 0, multi-comma trivia, AoT rendering).

No claim is made that these twelve cases are a random sample. They are the highest-survivor functions, selected to show actionable failure modes.

Source: week11\property\PROPERTY_CHECKPOINT.md

Track D specialised stream — property-based testing checkpoint

Group A · tomlkit · Week 11

Track D's specialised stream is property-based testing with Hypothesis. It was introduced in Weeks 6–7 and is summarised here because it is the source of two of the three TDD cycles in `week11/TDD_LOG.md`. This is not a new Week 11 deliverable; it is the completed Track D stream evidence for the final report.

Why property-based testing for tomlkit

tomlkit is a parser/serialiser. Its main risk is not a single known example but the input space: empty and malformed documents, unusual Unicode, all four TOML string forms, integer boundaries, timezone offsets, and nested documents. An example-based AI suite tests the examples the model thought of. Properties state invariants over generated inputs and let Hypothesis search for counterexamples.

Properties implemented

File: `tests_property/test_properties.py`. Each property runs 200 generated examples (Hypothesis `ci` profile).

ID	Property	Strategy / input domain	Why it matters
P1	`dumps(data)` then `parse(...).unwrap()` preserves the data	Recursive JSON-like docs with TOML scalars, lists and dicts	Serialise/parse contract
P2	An unmodified document re-serialises byte-identically	Documents produced by `dumps`	Style/format preservation
P3	`String.from_raw(s, kind)` round-trips, or literal forms reject invalid input	Arbitrary Unicode text × all `StringType` values	All four string forms, escaping and rejection
P4	`integer(n)` / `value()` round-trip signed 64-bit integers	`INT64` boundaries	Numeric boundaries and raw text
P5	`date()`, `time()`, `datetime()` accept exactly their own kind	Dates, times, datetimes with UTC/offset/naive zones	Factory contract and RFC 3339 handling
D7	Multiline strings never emit a bare CR; CRLF is preserved	Bare CR / CRLF strings in MLB and MLL	Regression for a real defect

Evidence

Command:

```
python -m pytest -q tests_property -v
```

Result on the Week 11 frozen commit:

```
10 passed in 6.15s
```

Environment: Python 3.13.14, pytest 9.1.1, Hypothesis 6.170.12.

Raw output: `week11/property/pytest_property.txt`.

What the stream found

Hypothesis produced two counterexamples that became real defects and TDD cycles:

1. **D1 — `date()` accepted a datetime string.** `datetime` is a subclass of `date`, and `api.date()` used `isinstance`. Property P5 exposed the contract violation. Fixed test-first in C1 (`9472f8a` → `a4f54db` → `fad6c12`).

2. **D7 — multiline strings emitted a bare CR.** Property P3 generated a string containing a lone `\r`; `String.from_raw` emitted it inside a multiline string, producing output that did not round-trip. Fixed test-first in C2 (`51c6863` → `a38a3d8` → `26421d6`).

D6 (`comment()` with a bare CR) was found through exploratory testing rather than PBT, and fixed in C3. Together these show the complementary value of property-based, example-based and exploratory testing.

Limitations

- Hypothesis is a search procedure, not a proof; 200 examples is a bounded sample, not exhaustive coverage.
- The document strategy generates valid Python data, not arbitrary TOML text; it does not exhaustively test the grammar or invalid-input space.
- Stateful testing (`RuleBasedStateMachine`) for repeated edits was not used; the Week 9 exploratory session covered repeated array edits manually.
- A passing property does not prove the absence of bugs; mutation testing measures how well the property suite detects seeded faults.

Link to Week 11

The TDD regression tests are part of the final hybrid suite measured in `week11/mutation_v2/`. The mutation report records the score improvement from the Week 7 AI-only baseline to the final suite and identifies the survivors that remain.

Final suite result

Source: week11\full_suite.txt

```
.....................................................................  [  4%]
.....................................................................  [  9%]
.....................................................................  [ 14%]
.....................................................................  [ 19%]
.....................................................................  [ 23%]
.....................................................................  [ 28%]
.....................................................................  [ 33%]
.....................................................................  [ 38%]
.....................................................................  [ 42%]
.....................................................................  [ 47%]
.....................................................................  [ 52%]
.....................................................................  [ 57%]
.....................................................................  [ 62%]
.....................................................................  [ 66%]
.....................................x..x..........................  [ 71%]
.....................................................................  [ 76%]
.....................................................................  [ 81%]
.....................................................x...............  [ 85%]
.....................................................................  [ 90%]
........x............................................................  [ 95%]
.......................x.....................................      [100%]
1502 passed, 5 xfailed in 26.43s
```

Property-test result

Source: week11\property\pytest_property.txt

```
============================= test session starts ==============================
platform linux -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0
rootdir: /home/zombie/tomlkit-w11
configfile: pyproject.toml
plugins: cov-7.1.0, hypothesis-6.170.12, bdd-9.0.0, platformdirs-4.13.5
collected 10 items

tests_property/test_properties.py ..........                             [100%]

============================== 10 passed in 6.94s ==============================
```

Selected survivor diffs

Source: week11\mutation_v2\selected_diffs.txt

```
===== tomlkit.parser.x|Parser|_parse_table__mutmut_10
# tomlkit.parser.x|Parser|_parse_table__mutmut_10: survived
--- tomlkit/parser.py
+++ tomlkit/parser.py
@@ -13,7 +13,7 @@
     self.inc()  # Skip opening bracket
```

```
     if self.end():
-        raise self.parse_error(UnexpectedEofError)
+        raise self.parse_error(None)

     is_aot = False
     if self._current == "[":

===== tomlkit.items.x|AoT|insert__mutmut_10
# tomlkit.items.x|AoT|insert__mutmut_10: survived
--- tomlkit/items.py
+++ tomlkit/items.py
@@ -3,7 +3,7 @@
     if not isinstance(value, Table):
         raise ValueError(f"Unsupported insert value type: {type(value)}")
     length = len(self)
-    if index < 0:
+    if index <= 0:
         index += length
     if index < 0:
         index = 0

===== tomlkit.container.x|Container|append__mutmut_109
# tomlkit.container.x|Container|append__mutmut_109: survived
--- tomlkit/container.py
+++ tomlkit/container.py
@@ -98,7 +98,7 @@
                             self._raw_append(key, item)

                         if validate:
-                            self._validate_out_of_order_table(key)
+                            self._validate_out_of_order_table(None)

                         return self


===== tomlkit.items.x|InlineTable|as_string__mutmut_102
# tomlkit.items.x|InlineTable|as_string__mutmut_102: survived
--- tomlkit/items.py
+++ tomlkit/items.py
@@ -64,7 +64,7 @@
             # not after any trailing comment/whitespace -- otherwise the
             # comma is swallowed by a trailing comment (see #512).
             buf = f"{buf[:last_value_end]},{buf[last_value_end:]}"
-            needs_separator = False
+            needs_separator = None

         v_trivia_trail = v.trivia.trail.replace("\n", "")
         if k.is_dotted() and isinstance(v, Table):

===== tomlkit.container.x|Container|_validate_table_candidate__mutmut_10
# tomlkit.container.x|Container|_validate_table_candidate__mutmut_10: survived
--- tomlkit/container.py
+++ tomlkit/container.py
@@ -8,7 +8,7 @@
             if isinstance(existing, (Table, AoT)) != isinstance(v, (Table, AoT)):
                 raise KeyAlreadyPresent(k)
             if k.is_dotted():
-                raise TOMLKitError("Redefinition of an existing table")
+                raise TOMLKitError("redefinition of an existing table")
             if isinstance(existing, Table) and isinstance(v, Table):
                 if not existing.is_super_table() and not v.is_super_table():
                     # Both sides are concrete `[table]` definitions of the

===== tomlkit.items.x|Array|__delitem____mutmut_101
# tomlkit.items.x|Array|__delitem____mutmut_101: survived
--- tomlkit/items.py
+++ tomlkit/items.py
@@ -51,7 +51,7 @@
                 for j in range(idx, len(self._value)):
                     group = self._value[j]
                     if group.indent is not None and "," in group.indent.s:
-                        group.indent._s = group.indent.s.replace(",", "", 1)
+                        group.indent._s = group.indent.s.replace(",", "", )
                         break
             if group_rm.indent is not None and "\n" in group_rm.indent.s:
                 # Restore the removed group's newline onto the next group

===== tomlkit.container.x|Container|_replace_at__mutmut_10
# tomlkit.container.x|Container|_replace_at__mutmut_10: survived
--- tomlkit/container.py
```

```
+++ tomlkit/container.py
@@ -18,7 +18,7 @@
     # keeping the dotted key duplicates the prefix onto the header (#524) and
     # lets the header swallow following dotted siblings (#542). Drop the dotted
     # key so the replacement renders as a plain table/array of tables.
-    dotted_to_header = k.is_dotted() and (
+    dotted_to_header = k.is_dotted() or (
         isinstance(value, AoT)
         or (isinstance(value, Table) and not value.is_super_table())
     )

===== tomlkit.items.x‖Array‖add_line__mutmut_11
# tomlkit.items.x‖Array‖add_line__mutmut_11: survived
--- tomlkit/items.py
+++ tomlkit/items.py
@@ -23,7 +23,7 @@
         ]
         """
     if comment and ("\n" in comment or "\r" in comment):
-        raise ValueError("Comment cannot contain line breaks")
+        raise ValueError("XXComment cannot contain line breaksXX")
     new_values: list[Item] = []
     first_indent = f"\n{indent}" if newline else indent
     if first_indent:

===== tomlkit.parser.x‖Parser‖_parse_string__mutmut_103
# tomlkit.parser.x‖Parser‖_parse_string__mutmut_103: survived
--- tomlkit/parser.py
+++ tomlkit/parser.py
@@ -76,7 +76,7 @@
             # try to process current as a closing delim
             original = self.extract()
```

Showing the first 120 of 167 lines. Full file: `week11\mutation_v2\selected_diffs.txt`.

Source: week11\mutation_v2\README.md

Week 11 mutation testing v2 — evidence

Track D · tomlkit · final hybrid suite

This folder contains the Week 11 re-run of mutation testing on the final project suite. The report is `mutation_v2_report.md`.

Scope

- Source under mutation: `tomlkit/`, excluding `tomlkit/_compat.py` (platform compatibility shim) and `tomlkit/_types.py` (covered by the separate Week 10 `_types.py` exercise). This keeps the v2 scope comparable to the Week 7 v1 baseline.
- Final hybrid suite: `tests/`, `tests_ai/`, `tests_manual/`, `tests_bdd/`, `tests_integration/`, `tests_property/`, `tests_tdd/`.
- Runner: mutmut 3.8.0 on Ubuntu (WSL2), Python 3.13.14, pytest 9.1.1.
- One test is deselected because mutmut's trampoline adds stack frames to a deliberately deeply nested parser input and triggers `RecursionError`: `test_parse_rejects_deeply_nested_documents`. The test still runs in the normal suite; it is not part of the mutation run.

Reproduce

```
# WSL2 / Ubuntu, from the repository root
python3 -m venv .venv && . .venv/bin/activate
pip install -e . pytest pytest-cov hypothesis pytest-bdd radon "mutmut==3.8.0"
git submodule update --init --recursive

# [tool.mutmut] used for the run:
# source_paths = ["tomlkit/"]
# tests_dir = ["tests/", "tests_ai/", "tests_manual/", "tests_bdd/",
#              "tests_integration/", "tests_property/", "tests_tdd/"]
# also_copy = [same directories]
# do_not_mutate = ["tomlkit/_compat.py", "tomlkit/_types.py"]
# pytest_add_cli_args = ["-p", "no:cacheprovider", "-k",
#                        "not test_parse_rejects_deeply_nested_documents"]

mutmut run --max-children 8
mutmut results --all true > week11/mutation_v2/mutmut_results_v2.txt
python week11/mutation_v2/scripts/summarise_v2.py \
  week11/mutation_v2/mutmut_results_v2.txt \
  week11/mutation_v2/mutation_v2_summary.json \
  week11/mutation_v2/v1_vs_v2.csv
```

Files

File	Purpose
mutation_v2_report.md	Report (Markdown source)
mutmut_results_v2.txt	Raw mutmut status for every mutant
mutation_v2_summary.json	Overall and per-module counts/scores
v1_vs_v2.csv	Week 7 v1 vs Week 11 v2 by module
survivors_v2.txt	Surviving mutant names
survivor_rootcause_v2.md	Root-cause analysis of selected survivors
scripts/summarise_v2.py	Builds the summary and comparison

Score formula

```
score = (killed + timeout) / total × 100

covered_score = (killed + timeout) / (total − no_tests) × 100
```

Timeouts are counted as detected, following the Week 7 convention. `no_tests` means the mutated line was never executed by the suite.

TDD C1 red

Source: week11\tdd_runs\c1_red.txt

```
============================ short test summary info ============================
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27T07:32:00]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27 07:32:00Z]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[0001-10-30 00:00]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27T07:32:00-08:00]
4 failed, 1 passed in 0.20s
```

TDD C1 green

Source: week11\tdd_runs\c1_green.txt

```
5 passed in 0.23s
1475 passed, 5 xfailed in 7.62s
```

TDD C1 refactor

Source: week11\tdd_runs\c1_refactor.txt

```
1475 passed, 5 xfailed in 7.57s
```

TDD C2 red

Source: week11\tdd_runs\c2_red.txt

```
============================ short test summary info ============================
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[a\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[\r]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[a\r\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[end\r]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_literal_rejects_bare_cr[a\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_literal_rejects_bare_cr[\r]
```

```
6 failed, 2 passed, 5 deselected in 0.19s
```

TDD C2 green

Source: week11\tdd_runs\c2_green.txt

```
13 passed in 0.29s
1485 passed, 3 xfailed in 6.79s
```

TDD C2 refactor

Source: week11\tdd_runs\c2_refactor.txt

```
1485 passed, 3 xfailed in 6.79s
```

TDD C3 red

Source: week11\tdd_runs\c3_red.txt

```
========================== short test summary info ===========================
FAILED tests_tdd/test_tdd_cycles.py::test_c3_comment_splits_every_line_break[a\rb-# a\n# b]
FAILED tests_tdd/test_tdd_cycles.py::test_c3_comment_splits_every_line_break[a\r\rb-# a\n#\n# b]
2 failed, 2 passed, 13 deselected in 0.17s
```

TDD C3 green

Source: week11\tdd_runs\c3_green.txt

```
17 passed in 0.17s
1489 passed, 3 xfailed in 6.63s
```

TDD C3 refactor

Source: week11\tdd_runs\c3_refactor.txt

```
1489 passed, 3 xfailed in 6.90s
```

The full 5,389-line v2 mutmut output is `week11/mutation_v2/mutmut_results_v2.txt`; survivor lists are `survivors_v2.txt` and `survivors_v1_v2.csv`. The chart workbook is `week11/Week11_TrackD_Findings.xlsx`.

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
week11\full_suite.txt	2 KB	
week11\mutation_v2\mutation_v2_report.md	14 KB	
week11\mutation_v2\mutation_v2_summary.json	1 KB	
week11\mutation_v2\mutmut_results_v2.txt	325 KB	
week11\mutation_v2\README.md	3 KB	
week11\mutation_v2\requirements.txt	163 B	
week11\mutation_v2\scripts\build_excel.py	17 KB	
week11\mutation_v2\scripts\compare_survivors.py	2 KB	
week11\mutation_v2\scripts\make_chart.py	1 KB	
week11\mutation_v2\scripts\pick_survivors.py	2 KB	
week11\mutation_v2\scripts\README.md	1 KB	
week11\mutation_v2\scripts\summarise_v2.py	4 KB	
week11\mutation_v2\selected_diffs.txt	6 KB	
week11\mutation_v2\selected_mutants.txt	638 B	
week11\mutation_v2\survivor_candidates.csv	15 KB	
week11\mutation_v2\survivor_rootcause_v2.md	6 KB	
week11\mutation_v2\survivors_v1_v2.csv	166 KB	[v1 baseline]
week11\mutation_v2\survivors_v2.txt	48 KB	
week11\mutation_v2\v1_vs_v2.csv	339 B	[v1 baseline]
week11\mutation_v2\v1_vs_v2.png	42 KB	[v1 baseline]
week11\property\PROPERTY_CHECKPOINT.md	4 KB	
week11\property\pytest_property.txt	457 B	
week11\README.md	2 KB	
week11\TDD_LOG.md	4 KB	
week11\tdd_runs\c1_green.txt	52 B	

File	Size	Tag
week11\tdd_runs\c1_red.txt	503 B	
week11\tdd_runs\c1_refactor.txt	33 B	
week11\tdd_runs\c2_green.txt	53 B	
week11\tdd_runs\c2_red.txt	632 B	
week11\tdd_runs\c2_refactor.txt	33 B	
week11\tdd_runs\c3_green.txt	53 B	
week11\tdd_runs\c3_red.txt	317 B	
week11\tdd_runs\c3_refactor.txt	33 B	
week11\Week11_TrackD_Findings.xlsx	28 KB	